

MÓDULO 3

DESARROLLO ORIENTADO A ESPECIFICACIONES (SDD) CON IA

Análisis, Arquitectura y Prototipado AI-First: De la ambigüedad al contrato ejecutable

¿Qué es el SDD (Spec-Driven Development)?

SPECIFICATION-DRIVEN DEVELOPMENT

Metodología emergente donde una especificación formal y legible por máquinas sirve como la fuente única de verdad. La IA no "adivina" el código; ejecuta un contrato estricto, reduciendo la deuda técnica y garantizando la trazabilidad.

El Problema: Vibe Coding vs. El Estándar SDD

✗ Vibe Coding

- Prompts vagos e improvisados ("Haz un clon de Jira").
- El código se vuelve la única fuente de verdad.
- Alta dependencia de las alucinaciones del LLM.
- Imposible auditar, mantener o escalar a largo plazo.

✓ Spec-Driven Development

- Contexto estructurado mediante Documentos de Requerimientos (PRD).
- Las especificaciones dirigen TODO el ciclo de vida.
- Arquitectura ejecutable (Diagrams-as-Code).
- Gobernanza clara sobre Agentes de IA autónomos.

El Impacto de Implementar SDD en 2026

>2x

MAYOR PRECISIÓN EN 1ER
INTENTO AL USAR UN PRD VS.
PROMPTS VAGOS (REPORTADO
POR EARLY ADOPTERS).

40-80%

REDUCCIÓN ESTIMADA DE
DEFECTOS EN PRODUCCIÓN CON
SHIFT-LEFT TESTING.

~80%

CÓDIGO ASISTIDO POR IA EN
EQUIPOS TECH LÍDERES
(ESTIMADO 2025-2026).

El Embudo de la Ambigüedad en Requerimientos

1. Stakeholders y Reuniones (Caos y Ruido)

Ideas a medias, contradicciones y requerimientos no verbalizados.

2. Análisis de IA Socrático

La IA cuestiona y obliga a definir roles, flujos límite y restricciones técnicas.

3. Contrato Ejecutable (BDD)

Reglas estrictas listas para ser codificadas sin lugar a "interpretación creativa".

El Flujo Maestro SDD de 6 Fases



1. Specify

Definir la intención, requerimientos y criterios de éxito mediante un PRD claro que sirva como única fuente de verdad.



2. Plan

Desglosar la especificación en fases de implementación, evaluando dependencias, arquitectura y posibles riesgos.



3. Design

Revisión técnica y validación de la arquitectura del sistema asegurando escalabilidad y seguridad antes de codificar.



4. Implement

Generación de código por agentes IA utilizando las especificaciones aprobadas como un contrato ejecutable estricto.



5. Test

Validación exhaustiva mediante pruebas generadas para verificar que el comportamiento cumpla exactamente la especificación.



6. Deploy

Lanzamiento a producción, configuración de infraestructura como código y establecimiento de monitoreo continuo.

“La persona que se comunice mejor será el programador más valioso del futuro. La nueva habilidad escasa es escribir especificaciones que capturen completamente tu intención y tus valores.”

 - Sean Grove, OpenAI (AI Engineer Conference)

Fase 1: Ingesta de Contexto y el PRD

PRODUCT REQUIREMENTS DOCUMENT (PRD)

El README definitivo del proyecto. Un documento en Markdown que define explícitamente Objetivos, In-Scope (Qué hacer), Out-Scope (Qué NO hacer para evitar el Scope Creep) y el Stack Tecnológico. Es el mapa mental del Agente de IA.

Prompt Engineering para Requerimientos



Asignación de Persona

Instruir al LLM a actuar como "Product Manager Senior y Arquitecto Cloud" mejora el nivel técnico de sus preguntas.



Chain-of-Thought (CoT)

Forzar al modelo a razonar paso a paso. Analizar transcripciones -> extraer problemas -> formular el documento.



Patrón "Document & Clear"

Para evitar el problema "Lost in the Middle" (pérdida de atención del LLM en el centro de contextos muy largos), se debe generar el archivo, guardarlo y reiniciar el chat.

El Caso de Estudio: Mini Jira

PROYECTO DEL MÓDULO

Durante este módulo construiremos un sistema de gestión de tickets (Mini Jira) usando SDD de principio a fin. Como punto de partida recibirás el archivo `reunion_mini_jira.txt`: una transcripción real de reunión con stakeholders, llena de requerimientos vagos, contradictorios e incompletos. Tu misión es convertir ese caos en un contrato ejecutable paso a paso, usando IA.

Actividad 1: La Ingesta Socrática

1

Análisis de Contexto (Gemini)

Sube el archivo `reunion_mini_jira.txt` a Gemini (soporta gran ventana de contexto).

2

Prompt Consultivo

Ejecuta: *"Actúa como PM Senior. Analiza la transcripción adjunta y hazme 3 preguntas exhaustivas sobre permisos, datos y concurrencia que los stakeholders omitieron."*

3

Generación del Contrato

Responde a la IA tomando las decisiones técnicas. Luego pide: *"Consolida las respuestas en un PRD (specs.md) con In-Scope, Out-Scope y Stack Tecnológico."*

4

Document & Clear

Copia el resultado, guárdalo localmente como `specs.md` y **cierra el chat**.

Fase 2: Behavior-Driven Development (BDD)

SINTAXIS GHERKIN (GIVEN-WHEN-THEN)

Formato estandarizado que transforma reglas de negocio en especificaciones ejecutables. Conecta las historias de usuario con pruebas automatizadas que las inteligencias artificiales entienden a la perfección.

Criterios: Imperativo vs. Declarativo

✗ Criterios Imperativos (Malo)

- "Cuando hago clic en el botón con ID #submit-btn".
- Las pruebas se vuelven frágiles.
- Si la UI cambia, el test falla.
- Enfocado en CÓMO se hace.

✓ Criterios Declarativos (Bueno)

- "Cuando envío mis credenciales de acceso".
- Resistente a rediseños de UI.
- Valida la lógica de negocio real.
- Enfocado en QUÉ se logra.

Anatomía de un Escenario Gherkin

CLÁUSULA	PROPÓSITO ARQUITECTÓNICO	EJEMPLO: GESTIÓN DE TICKETS
Given (Dado)	Establece el estado previo (Precondiciones y BD)	<i>Given</i> que soy un Administrador autenticado.
When (Cuando)	La acción o evento que dispara la prueba.	<i>When</i> muevo un ticket de "To-Do" a "Done".
Then (Entonces)	El resultado observable o cambio en el sistema.	<i>Then</i> el ticket actualiza su timestamp de cierre en la BD.

Shift-Left Testing y Edge Cases



Descubrimiento Autónomo

A partir de un Happy Path, los LLMs pueden inferir Casos Límite (ej. entradas masivas, pérdida de red) antes de codificar.



Shift-Left Real

Al tener las pruebas BDD redactadas, convertimos QA en prevención proactiva en lugar de detección reactiva post-despliegue.



Trazabilidad Total

Cada historia de usuario se convierte en un contrato que la IA evaluará como "Pass/Fail" en la fase de implementación.

AI Shift-Left Testing

PRUEBAS BASADAS EN ESPECIFICACIONES

Es la práctica de adelantar (Shift-Left) la creación del plan de pruebas a la fase de diseño. En lugar de esperar al código, los agentes de IA analizan el PRD y los criterios Gherkin para generar escenarios de prueba, atrapando errores conceptuales cuando cuestan \$1 en lugar de \$10,000 en producción.

El Superpoder de la IA en QA

De la especificación a la validación predictiva



Generación Spec-Driven

La IA traduce la complejidad del PRD en escenarios de prueba listos para usarse, eliminando la ambigüedad de la interpretación manual.



Detección de Edge Cases

Los LLMs infieren de forma autónoma condiciones límite, transiciones de estado complejas y rutas negativas que los humanos suelen omitir.



Trazabilidad Total

Mapeo automático entre los requerimientos de negocio y los casos de prueba, garantizando una cobertura del 100% del contrato.

Matriz de Priorización de Edge Cases

Cómo indicarle a la IA qué pruebas generar primero

IMPACTO DE NEGOCIO	PROBABILIDAD DE OCURRENCIA	ACCIÓN / PRIORIDAD QA
Alto (Ej. Fallo en pasarela de pagos)	Alta	Crítica: Generar tests exhaustivos inmediatamente.
Alto (Ej. Corrupción de BD)	Baja (Ej. Año bisiesto)	Alta: Probar en cuanto existan recursos.
Bajo (Ej. Falla estética en UI)	Alta	Media: Cobertura de tests automatizados básicos.
Bajo	Baja	Baja: Pruebas opcionales / esfuerzo mínimo.



Actividad Integrada: Historias de Usuario y Plan de Pruebas



1

Anclaje de Contexto

Inicia un **nuevo chat** y adjunta tu archivo `specs.md` creado anteriormente.

2

Generar Historias de Usuario (Gherkin)

"Actúa como Product Owner. Basado en el PRD, redacta las 3 Historias de Usuario críticas para el MVP usando formato BDD Gherkin declarativo."

3

Identificar Edge Cases

"Deduce 2 Edge Cases o escenarios de fallo (ej: edición concurrente, entradas nulas). Escríbelos también en Gherkin."

4

Guardar Backlog

Guarda el resultado en `backlog.md` con las historias de usuario validadas.

5

Generar Plan de Pruebas

"Actúa como QA Lead Senior. Genera un Plan de Pruebas enfocado en los criterios de aceptación Gherkin del backlog anterior."

6

Casos Límite Críticos

"Identifica 5 Edge Cases críticos (fallas de red, entradas nulas, concurrencia extrema) que puedan romper el MVP. Usa matriz de priorización de riesgo."

7

Guardar Plan de Pruebas

Guarda el resultado como `test_plan.md`. Este será el mapa de validación para la etapa de desarrollo.

Fase 3: Arquitectura y Diagrams as Code

MERMAID.JS

Un lenguaje basado en Markdown para generar diagramas (Flujo, C4, Secuencia, ERD) dinámicamente. Permite que la Arquitectura Software se versione en Git y sea fácilmente consumida y generada por Modelos de IA.

La Guerra de la Eficiencia de Tokens

Herramientas Visuales (Draw.io / XML)

- Alta redundancia y metadatos visuales inútiles para IA.
- Consume **cientos de tokens** por diagrama básico.
- Control de versiones (Git Diffs) caótico y roto.
- Mala afinidad para ser interpretado o editado por un LLM.

Mermaid.js (Texto Estructurado)

- Sintaxis declarativa de alta densidad semántica.
- Consume solo **decenas de tokens** por diagrama.
- Git Diffs limpios (solo cambia la lógica, no coordenadas).
- *Trade-off aceptado:* Cruce automático de líneas a cambio de máxima eficiencia AI.

El Ecosistema Pair Architecture



Contrato Ejecutable

PRD (specs.md)

Fuente única de verdad



C4 Containers

(HLD)

Alto nivel



Arquitecto Humano

Ajustes

Orquestación



Agente IA

Generador Mermaid

Automatización



Diagrama Secuencia

(LLD)

Bajo nivel



Actividad 3: Diagramando HLD y LLD en Mermaid

1

Contextualización (Solo Specs)

Inicia un nuevo chat, provee a la IA el archivo `specs.md` y `backlog.md`.

2

High-Level Design (C4)

Prompt: *"Actúa como Arquitecto de Software. Genera un modelo C4 a nivel de contenedores para el Mini Jira usando sintaxis estricta Mermaid.js."*

3

Low-Level Design (Secuencia)

Prompt: *"Ahora genera un diagrama de secuencia en Mermaid mostrando el flujo de la historia: 'Mover ticket de To-Do a Done'. Incluye la interacción Front, API, y BD."*

4

Validación Visual

Copia el código en `mermaid.live` o en tu IDE (VS Code con extensión). Guarda el resultado en `architecture.mermaid`.

Fase 4: Decisiones Arquitectónicas

ARCHITECTURE DECISION RECORDS (ADRS)

Documentos inmutables que capturan las decisiones arquitectónicas importantes junto con su contexto, alternativas evaluadas y consecuencias. Con IA, el ADR evoluciona de ser "tarea manual pesada" a "Event Sourcing" automático extraído de conversaciones técnicas.

Anatomía de un ADR Asistido por IA



Contexto (Problema)

La IA resume el reto de negocio y las restricciones que forzaron a tomar una decisión de diseño específica.



Opciones Consideradas

Análisis de compensaciones (trade-offs) listando pros y contras de cada tecnología (ej. SQL vs NoSQL).



Decisión Tomada

Selección clara y justificada. La "Semilla de Verdad" que la IA documentará de forma inmutable.



Consecuencias (Impacto)

¿Qué se hace más fácil? ¿Qué deuda técnica asumimos? La IA simula escenarios futuros en base a la elección.

ADRs como Event Sourcing en la Era IA





Actividad 4: Documentando Decisiones Arquitectónicas

1

Alimentando al LLM

Usa la información previa del Mini Jira o provee un resumen indicando que se evaluaron opciones de Backend-as-a-Service para el MVP.

2

Prompt Metaprompting (ADR)

"Genera un documento ADR en Markdown detallando la decisión de usar Supabase en lugar de Firebase para nuestro MVP de Mini Jira. Incluye Contexto, Opciones, Decisión y Consecuencias."

3

Validación Humana

Revisa que la IA no haya inventado requerimientos inexistentes en el PRD.

4

Cierre

Guarda el archivo como `001-database-selection.md` en una carpeta de ADRs.

Fase 5: Modelado de Datos Schema-First

1. Cimientos (Base de Datos)

1

Definir entidades, relaciones e integridad referencial usando IA asegura que el backend futuro tenga un mapa inmutable que no pueda alucinar.

2. Paredes (Capa API REST/GraphQL)

2

Las interfaces se construyen estrictamente exponiendo los contratos del Schema definido.

3. Decoración (Frontend UI)

3

Se diseña iterativamente consumiendo la estructura y Mock Data, completamente aislada del backend.

Beneficios de los Generadores de Esquema IA



Velocidad Estructural

Traducción de lenguaje natural (PRD) a DDL SQL en segundos. La IA infiere llaves foráneas y tipos de datos automáticamente.



Mock Data Realista

Generación de 'Seed Data' con contexto. Adiós al "Lorem Ipsum". Hola a registros con nombres, emails y estados de tickets reales.



Constraints Proactivos

La IA, guiada por el ADR, aplica índices, borrado en cascada y validaciones (ej. checks de longitud de contraseña) directamente en BD.

Bases de Datos e Inferencia IA

TIPO DE BASE DE DATOS	CASO DE USO IDEAL	SALIDA GENERADA POR LA IA
PostgreSQL / MySQL	Apps de producción, queries complejos, consistencia estricta.	DDL con Tablas, Foreign Keys, Checks y Constraints de integridad.
MongoDB / CouchDB	Evolución rápida, datos flexibles.	Estructuras de documentos JSON, colecciones y esquemas anidados.
Neo4j	Relaciones profundas, recomendaciones, grafos.	Nodos Cypher, etiquetas, relaciones y propiedades conectadas.



Actividad 5: Modelado de Datos en Supabase

1

Contexto Base

Proporciona a la IA tu `specs.md` y tu `architecture.mermaid` como contexto del modelo de datos.

2

Prompt ER Diagram

"Genera un diagrama Entidad-Relación en Mermaid (erDiagram) para el Mini Jira. Incluye entidades Usuarios, Tickets y sus relaciones con cardinalidad."

3

Prompt Schema Supabase

"Basado en el ER anterior, genera el SQL para crear estas tablas en Supabase (PostgreSQL). Incluye RLS políticas básicas y timestamps con timezone."

4

Prompt Seed Data

"Añade comandos INSERT para poblar Supabase con Mock Data realista: 3 usuarios y 5 tickets en estados To-Do, In-Progress y Done."

5

Guardar Activos

Guarda el ER como `er_diagram.mermaid` y el SQL como `init_db.sql` para ejecutarlo en el SQL Editor de Supabase.

Fase 6: Prototipado UI Acelerado por IA

PROTOTIPADO AI-POWERED 2026

El proceso de pasar de un wireframe, imagen o PRD a componentes interactivos en React/Tailwind en minutos. Se enfoca en validar flujos de usuario con código de interfaz real antes de programar integraciones de Backend complejas.

Atomic Design Dirigido por IA

1. Átomos y Moléculas

1

Comienza pidiendo a la IA componentes pequeños e independientes: un botón, una tarjeta de tarea (TaskCard), un input de búsqueda. Restringe el diseño con Design Tokens.

2. Organismos

2

Pide combinar las moléculas en componentes más grandes: Una columna Kanban que filtra las tarjetas por estado usando el Mock Data de la BD.

3. Templates / Páginas

3

Ensambla los organismos. La IA junta 3 columnas Kanban en el Board general. Esta divulgación progresiva evita que el modelo de IA se abrume y alucine diseños rotos.

Control de Consistencia UI



Design Tokens

En el PRD, especifica paletas fijas (ej. Tailwind slate-900). Evita que la IA genere "50 tonos de azul diferentes" sin un diseñador humano.



Librerías Base (shadcn/ui)

Restringe a la IA a utilizar librerías estandarizadas accesibles (ARIA) en lugar de escribir CSS crudo desde cero, garantizando mantenibilidad.



Mock Data Inyección

El código UI generado debe recibir objetos JSON compatibles con la BD, logrando un prototipo que "se siente real" en las demostraciones a stakeholders.

El Paisaje de Herramientas UI (2026)

HERRAMIENTA AI	FORTALEZA PRINCIPAL	IDEAL PARA...
Google Stitch (Labs)	Genera UI y código frontend desde texto/bocetos usando modelos Gemini (Flash, Pro).	Validación temprana de diseño y generación rápida de pantallas únicas estáticas
v0.app (Vercel)	Componentes React precisos y código frontend con shadcn/ui.	Diseño visual a nivel de componente y hand-off a developers frontend.
Lovable / Bolt.new	Orquestación Full-Stack. Conecta front, back y DB al instante.	MVPs vivos para startups, pruebas de concepto operativas con lógica.
Replit	IDE en la nube potente con IA nativa y base de datos real.	Prototipos lógicos complejos que exigen backend en Python/Node.

Generadores de UI vs Orquestadores

Generadores Frontend (Stitch, v0)

- Generan vistas estáticas y componentes visuales en minutos a partir de prompts o imágenes.
- Ideales para validar el "Look & Feel" de una pantalla única con los stakeholders.
- Perfectos para iterar el diseño antes de invertir tiempo en lógica e integraciones.

Orquestadores (Lovable, Taskade)

- Entregan un MVP básico completo (Base de Datos + Agentes + APIs + UI).
- Conectan las capas del sistema de forma automatizada.
- Menor control granular visual sobre el código y estilos comparado con un generador frontend puro.



Actividad 6: Primer Vistazo con Google Stitch

Validaremos el diseño generando la pantalla principal hoy. El prototipado completo y backend será en la próxima sesión.

1

Ingreso a Google Stitch

Abre la herramienta en Google Labs. Pega el contenido de tu `specs.md` como contexto crítico inicial para establecer el objetivo.

2

Selección del Modelo Base

Configura Stitch para usar **Gemini 3 Flash** para obtener código y estilos precisos con alta velocidad de generación.

3

Prompt de Pantalla Única

"Genera únicamente la vista principal estática (Dashboard) del Mini JIRA. Muestra un tablero Kanban con las columnas To-Do, In Progress y Done, pobladas con datos de ejemplo."

4

Validación y Resguardo

Evalúa visualmente la interfaz generada por Stitch. Exporta el código o guarda la vista; será nuestra base para la implementación funcional en el módulo de desarrollo.

El Arsenal del Arquitecto IA

specs.md

PRD: El contrato ejecutable. Define alcance y reglas de negocio para los Agentes IA que codificarán luego.

backlog.md

Historias de Usuario (Gherkin BDD) y Edge Cases mapeados.

architecture.mermaid

Diagramas HLD/LLD eficientes en tokens y controlables en Git.

init_db.sql

Modelado Schema-First con Mock Data, listo para poblar un contenedor.

prototype.tsx

UI interactiva validada, construida incrementalmente (Átomos -> Páginas).

Orquestación Multi-Agente: El Futuro del SDD



Arsenal SDD

Contrato Ejecutable

Fuente única de verdad



Agente Arquitecto

Evalúa HLD

Diseño estructural



Agente Developer

Escribe Lógica

Implementación



Agente QA

Verifica BDD

Calidad y Seguridad



Humano Supervisor

Orquestación

Shift-Left

La Evolución de tu Rol como Ingeniero



Orquestador de Intenciones

Ya no eres un "picador de código". Te conviertes en el director de orquesta que traduce requerimientos vagos a especificaciones formales.



Custodio de la Calidad

Al aplicar Testing Shift-Left, tu deber es revisar implacablemente al IA, no asumir que siempre está en lo correcto.



Pensamiento Sistémico

Delegas el "Cómo" (la sintaxis) a la máquina, y tomas control total sobre el "Qué" y el "Por qué" (arquitectura, ADRs y valor de negocio).

¿PREGUNTAS O DUDAS?

¡GRACIAS POR SU PARTICIPACIÓN!